ECE 590
Neural-Network-Based Large Language Models

Tuesdays and Thursdays 1:25-2:40
Fitzpatrick Schiciano A 1464

ECE 590
Neural-Network-Based Large Language Models

Lecture 22
04/06/2026

Data on a Graph

- There are many problems of interest for which we have data that resides on a graph

- Social networks are a common example, with each individual a vertex, and edges existing between all connected vertices (many not connected)

- In some cases, the edges have binary weights, representing present or not, and in other cases there are real-valued weights associated with each edge, reflecting strength of connectivity

- Molecules are another important example, where the vertices may be atoms, and the network is defined by the presence/absence of connections between vertices

- In the context of molecules, may wish to predict their properties and possible molecular interactions (protein-protein interactions)

Transformers for Graphs

- Much recent work on employing neural networks for analysis of graphs

- One example is a model term a Graph Convolutional Network (GCN) and more generally a Graph Neural Network (GNN)

- Recent interest in exploring the potential of Transformers for analysis of graph/network data

- The Transformer must account for features that may exist connected to the vertices, while also accounting for properties of edges

- In language and vision, we have features connected with each position/block, but not inter-position and inter-block (edge) features

- Requires some modification to the Transformer setup to account for node *and* edge features

Positional Encoding on Graphs

- A key component of the Transformer is positional encoding, which is relatively simple for linear data

- For a graph it is more complicated, as the position of a vertex (node) must be defined in terms of all of its neighbors, not just to the “right” and “left” as in the linear setting

- Seek positional embeddings that capture the relative positions of all graph nodes

- The Graph Laplacian is a powerful tool for encoding relative position on a graph

Graph Laplacian

- Assume that we have a network composed of N nodes or vertices

- Assume that the matrix $A \in R^{N \times N}$ is the adjacency matrix, representing the strength of connection between node i and node j

- Specifically, element A_{ij} represents the degree of connectivity (edge strength) between vertex i and vertex j

- Introduce self-connections as

$$\tilde{A} = A + I_N$$

Random Walk on a Graph

$$\tilde{A} = A + I_N$$

- We define the total connectivity of node i to all other nodes, including itself:

$$\widetilde{D}_{ii} = \sum_{j=1}^{N} \tilde{A}_{ij}$$

- $\widetilde{D} = \mathrm{diag}(\widetilde{D}_{11}, \dots, \widetilde{D}_{NN})$ is a diagonal matrix, used for normalizing the rows of $\tilde{A}$:

$$W = \widetilde{D}^{-1}\tilde{A}$$

where row i may be seen as reflecting the probability of "walking" from node i to node j in a single step

Random Walk on a Graph

$$W = \widetilde{D}^{-1}\tilde{A}$$

- Row i may be seen as reflecting the probability of "walking" from node i to node j in a single step
- The probability of walking from node i to node j in $n \geq 1$ steps is reflected by row i of W^n
- Matrix W captures the interrelationships of all nodes on the graph
- Wish to use the eigenvectors of W as a measure of interrelationships of nodes
- Problem: W is asymmetric (in general), which means it has different left and right eigenvectors

Symmetric Graph Laplacian

- To manifest a symmetric graph matrix with which to consider eigenvectors, we consider

$$H = \widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$$

- Similar to earlier form of normalization, but now H is symmetric, and has a single set of eigenvectors

- Consider the eigenvectors and eigenvalues

$$H\phi = \lambda\phi$$

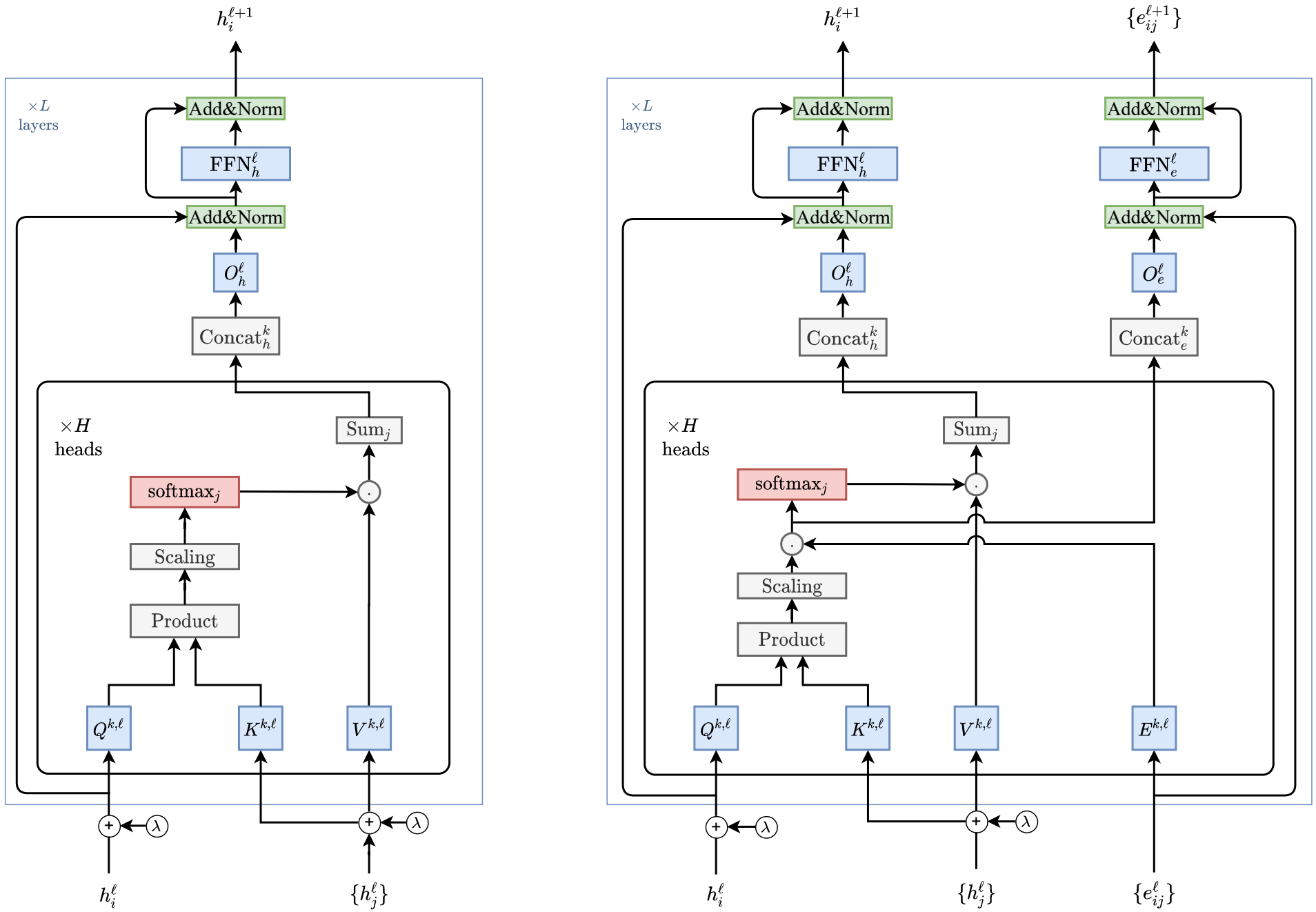
where $\lambda \in R$ and $\phi \in R^N$

- $\phi(i)$ and $\phi(j)$ are components i and j of eigenvector ϕ; expect $|\phi(i) - \phi(j)| \to 0$ for proximate nodes i and j

Laplacian Eigenvectors as Positional Encodings

$$H = \widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$$

- Let $\phi_1, \dots, \phi_m$ represent the m eigenvectors with largest respective eigenvalues $\lambda_1, \dots, \lambda_m$
- Use the m-dimensional vector $(\phi_1(i), \dots, \phi_m(i))$ as a positional encoding (PE) for node i
- With these PEs, the Transformer readily (and naively) translates for network data



Graph Transformer Layer

λ *Laplacian EigVecs as Positional Encoding*

Graph Transformer Layer with edge features

Accounting for Edge Features

- The graph Laplacian leverages eigenvectors related to the connectivity of the network

- There may be features or labels connected with each node, and these may be represented as node-dependent features, like in the usual Transformer

- There may also be edge-dependent features that we may wish to model and leverage

- Important for attention: The degree of attention between nodes i and j should be connected to the degree of feature alignment (including Laplacian-based PEs) as in traditional Transformer, but should also leverage edge information between nodes i and j

More Details

- The output of the hidden unit for node i at layer $l+1$ is analogous to traditional Transformer:

$$\hat{h}_i^{l+1} = O^l \bigvee_{k=1}^{H} \left(\sum_{j \in V} w_{ij}^{k,l} \, W_V^k h_j^l \right)$$

Concatenate outputs from H heads

- Use edge information to define attention weights:

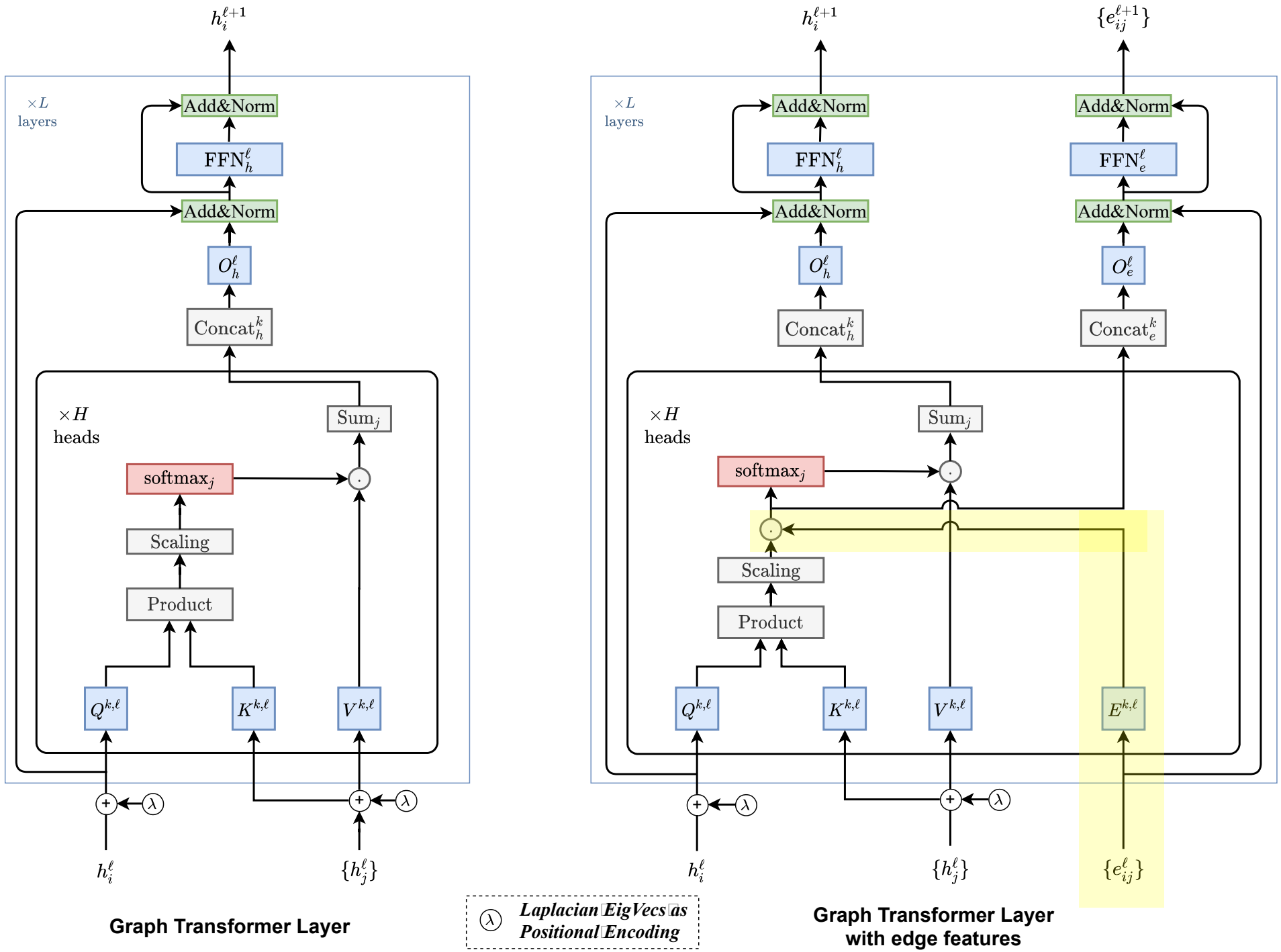
Three-vector inner product

$$\hat{w}_{ij}^{k,l} = W_Q^{k,l} h_i^l \circ W_K^{k,l} h_j^l \circ W_E^{k,l} e_{ij}$$

$$w_i^{k,l} = \mathrm{Softmax}[\hat{w}_{i1}^{k,l}, \dots, \hat{w}_{iN}^{k,l}]$$

Edge (i,j)

Rethinking graph transformers with spectral attention
D Kreuzer, D Beaini, W Hamilton… - Advances in neural …, 2021 - proceedings.neurips.cc
In recent years, the Transformer architecture has proven to be very successful in sequence processing, but its application to other data structures, such as graphs, has remained limited …
Save Cite Cited by 942 Related articles



Graph Transformer Layer

Graph Transformer Layer with edge features

Graph Transformer Pre-Training

- In GPT we pre-train by considering a task that may be done exhaustively with limited labeled data: predict the next word

- With BERT, we similarly consider task of predicting masked token and classifying whether two sentences are consecutive in text

- Fine-tune for some subsequent task, such as Q&A and/or RL-based alignment

- Can consider similar pre-training tasks for graph Transformer
  - Mask nodes like BERT
  - Add noise to edge features, and predict the noise-free version

Graph Transformer Down-Stream Tasks

- After we pretrain the graph Transformer on massive quantity of network data, fine-tune for desired subsequent task

- For example, predict properties of a molecule, which is represented as a network

- Could also consider binding properties of multiple molecules

- Challenge of graph Transformer: In general, there are N^2 edges e_{ij}

- Updating and storing all edge information is a significant computational bottleneck

Connections to Other Graph Neural Networks

Graph Convolutional Network

- Recall the symmetric graph Laplacian

$$H = \widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$$

- Because of the symmetric form of this operator, we can write

$$\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2} = U\Lambda U^T$$

where the columns of U represent the eigenvectors of the operator, and Λ is a diagonal matrix with the corresponding eigenvalues

- Consider a vector $x \in R^N$ with components associated with the N nodes of a graph

Fourier Representation of a Convolution

$$\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2} = U\Lambda U^T$$

- The operation $U^T x$ corresponds to mapping x into the graph Fourier domain, characterized by the operator eigenvectors (columns of U)

- The operation $\Lambda U^T x$ corresponds to multiplying the Fourier components of the spectral form of x by the diagonal elements of Λ, which is the Fourier form of the operator $\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$

- Finally, the representation

$$\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}x = U\Lambda U^T x$$

corresponds to an inverse Fourier transform back to the original domain of x

Fourier Representation of a Convolution

$$\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}x = U\Lambda U^T x$$

- We are here multiplying the Fourier transform of x by the Fourier components of the operator

- Multiplication in the Fourier domain corresponds to convolution in the original domain

- Because of the symmetric form of the symmetric normalized graph Laplacian, we can see each row of $\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$ as a shifted form of the same underlying signal/vector, consistent with a convolution

- We can view $\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}x$ as a convolution of x on a graph, with convolution connected to the rows of the normalized graph Laplacian

Relationship to Attention

$$\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}x = U\Lambda U^T x$$

- After performing the operation

$$y = \widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}x$$

every component of y is (obviously) a linear combination of the components of x, with each component represented by a shifted form of the same filter (convolution)

- Generalizes

$$y = \widetilde{D}^{-1}\tilde{A}x$$

which is exactly like attention, but does not correspond to convolution because $\widetilde{D}^{-1}\tilde{A}$ is asymmetric

Generalization of the Operator

- Consider the generalization

$$\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}x\Theta = U\Lambda U^{T}x\Theta$$

where $\Theta \in R^{N\times N}$

Generalization of the Operator

- Consider the generalization

$$\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}x\Theta = U\Lambda\underbrace{U^T x}_{\text{Graph Fourier transform of } x}\Theta$$

where $\Theta \in R^{N\times N}$

Generalization of the Operator

- Consider the generalization

$$\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}x\Theta = U\Lambda\underbrace{U^T x}_{\text{Graph Fourier transform of } x}\Theta$$

where $\Theta \in R^{N\times N}$

Linear transformation in Fourier space

Generalization of the Operator

- Consider the generalization

$$\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}x\Theta = \overbrace{U\underbrace{\Lambda\underbrace{U^T x}_{\text{Graph Fourier transform of } x}}_{\text{Linear transformation in Fourier space}}\Theta}^{\text{Multiply by spectrum of operator}}$$

where $\Theta \in R^{N\times N}$

Generalization of the Operator

- Consider the generalization

Inverse Fourier transform

Multiply by spectrum of operator

$$\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}x\Theta = U\Lambda U^T x\Theta$$

Graph Fourier transform of x

where $\Theta \in R^{N\times N}$

Linear transformation in Fourier space

Generalization of the Operator

- Consider the generalization

Inverse Fourier transform

Multiply by spectrum of operator

$$\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}x\Theta = U\Lambda U^T x\Theta$$

Graph Fourier transform of x

where $\Theta \in R^{N\times N}$

Linear transformation in Fourier space

- This may be seen a generalization of the kernel connected to the Fourier transform

Generalization of the Operator

- Consider the generalization

Inverse Fourier transform

Multiply by spectrum of operator

$$\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}x\Theta = U\Lambda U^T x\Theta$$

Graph Fourier transform of x

where $\Theta \in R^{N\times N}$

Linear transformation in Fourier space

- This may be seen a generalization of the kernel connected to the Fourier transform

- Now generalize to an F-dimensional feature vector at each position, $X \in R^{N\times F}$

Generalization for Vector Data on a Graph

- Considering now an F-dimensional vector at each node of the graph, the generalized convolutional operation is

$$\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}X\Theta = U\Lambda U^T X\Theta$$

- The matrix $\Theta \in R^{N\times N}$ may be seen as playing a role like W_V in an attention model

Generalization for Vector Data on a Graph

- Considering now an F-dimensional vector at each node of the graph, the generalized convolutional operation is

$$\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}X\Theta = U\Lambda U^T X\Theta$$

- The matrix $\Theta \in R^{N\times N}$ may be seen as playing a role like W_V in an attention model

- The rows of $\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$ play a role like the key-query attention in a Transformer

- The connection is clearer with the symmetric $\widetilde{D}^{-1}\tilde{A}$ but then we lose the connection to operating in the Fourier space U

Graph Convolutional Network

- The operation

$$\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}X\Theta = U\Lambda U^T X\Theta$$

may be seen as a linear operation of the data (rows of X) situated on the nodes of a graph

- Can take the output of this operation, and perform pointwise nonlinearities, like with a neural network

$$\mathrm{ReLu}\left(\hat{A}X\Theta\right)$$

$$\hat{A} = \widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$$

Graph Convolutional Network

- The operation

$$\widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}X\Theta = U\Lambda U^T X\Theta$$

may be seen as a linear operation of the data (rows of X) situated on the nodes of a graph

- Can take the output of this operation, and perform pointwise nonlinearities, like with a neural network

- For example, consider a two-layer model, for which at the output we wish to predict categorical observations

$$Z = \mathrm{softmax}[\hat{A}\ \mathrm{ReLu}\left(\hat{A}XW^{(0)}\right)W^{(1)}]$$

$$\hat{A} = \widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$$

Graph Convolutional Network

$$Z = \mathrm{softmax}[\hat{A}\ \mathrm{ReLu}\left(\hat{A}XW^{(0)}\right)W^{(1)}]$$

$$\hat{A} = \widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$$

- The operation $\hat{A}$ is defined by the connectivity across the graph

- The matrices $W^{(0)}$ and $W^{(1)}$ are to be learned, and can be interpreted as filter operations connected to the convolutional operations

- Each row of the N rows of Z corresponds to a probability vector over categories (C dimensional if there are C categories.

Relationship to Transformer

$$Z = \mathrm{softmax}[\hat{A}\ \mathrm{ReLu}\left(\hat{A}XW^{(0)}\right)W^{(1)}]$$

$$\hat{A} = \widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$$

- Can be viewed as a specific, simplified form of the Transformer
- Do not have explicit keys and queries, as the key-query attention weighting defined by rows of $\hat{A}$
- Since $\hat{A}$ is symmetric, can be viewed as a convolutional operation, with spectral (Fourier) character linked to the structure of a graph
- Learned matrices $W^{(0)}$ and $W^{(1)}$ play a role like value matrices at each of the two layers

GCN Application: Semi-Supervised Learning

$$Z = \mathrm{softmax}[\hat{A}\ \mathrm{ReLu}\left(\hat{A}XW^{(0)}\right)W^{(1)}]$$

$$\hat{A} = \widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$$

- Assume we are given a graph, with associated graph Laplacian (connectivity) $\hat{A}$

- At node i we have a feature vector $x_i \in R^F$

- For a subset of nodes $Y_L \subset \{1, \dots, N\}$ we have observed categorical data, for C categories

$$y_i \in \{1, \dots, C\}, \qquad i \in Y_L$$

GCN Application: Semi-Supervised Learning

$$Z = \mathrm{softmax}[\hat{A}\ \mathrm{ReLu}\left(\hat{A}XW^{(0)}\right)W^{(1)}]$$

$$\hat{A} = \widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$$

- Given:

  $\hat{A}$ (graph structure)
  $x_i \in R^F, i = 1, \dots, N$ (features at each node)
  $y_i \in \{1, \dots, C\}, i \in Y_L$ (partial set of labels, connected to subset of nodes

- Goal: Predict categories for all unlabeled nodes

$$p(Y_i = c|\mathrm{Context}) = z_{i,c}, \qquad c = 1, \dots, C, \qquad i \notin Y_L$$

- Procedure: Learn $W^{(0)}$ and $W^{(1)}$ based on a fit to the subset of observed categorical observations

GCN Application: Semi-Supervised Learning

$$Z = \text{softmax}[\hat{A}\ \text{ReLu}\left(\hat{A}XW^{(0)}\right)W^{(1)}]$$

$$\hat{A} = \widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$$

- Given:

  $\hat{A}$ (graph structure)
  $x_i \in R^F, i = 1, \dots, N$ (features at each node)
  $y_i \in \{1, \dots, C\}, i \in Y_L$ (partial set of labels, connected to subset of nodes

- For $i \in Y_L$, consider component $z_{i,y_i}(W^{(0)}, W^{(1)})$ of Z, which is the probability of the observed categorical outcome at node i, which we write explicitly as a function of the unknown matrices

GCN Application: Semi-Supervised Learning

$$Z = \text{softmax}[\hat{A}\ \text{ReLu}\left(\hat{A}XW^{(0)}\right)W^{(1)}]$$

$$\hat{A} = \widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$$

- Given:

  $\hat{A}$ (graph structure)
  $x_i \in R^F, i = 1, \dots, N$ (features at each node)
  $y_i \in \{1, \dots, C\}, i \in Y_L$ (partial set of labels, connected to subset of nodes

- For $i \in Y_L$, consider component $z_{i,y_i}(W^{(0)}, W^{(1)})$ of Z, which is the probability of the observed categorical outcome at node i, which we write explicitly as a function of the unknown matrices

- Learning objective:

$$\widehat{W}^{(0)}, \widehat{W}^{(1)} = \text{argmin}_{W^{(0)},W^{(1)}} \ -\sum_{i \in Y_L} \log z_{i,y_i}(W^{(0)}, W^{(1)})$$

GCN Application: Semi-Supervised Learning

$$Z = \mathrm{softmax}[\hat{A}\ \mathrm{ReLu}\left(\hat{A}XW^{(0)}\right)W^{(1)}]$$

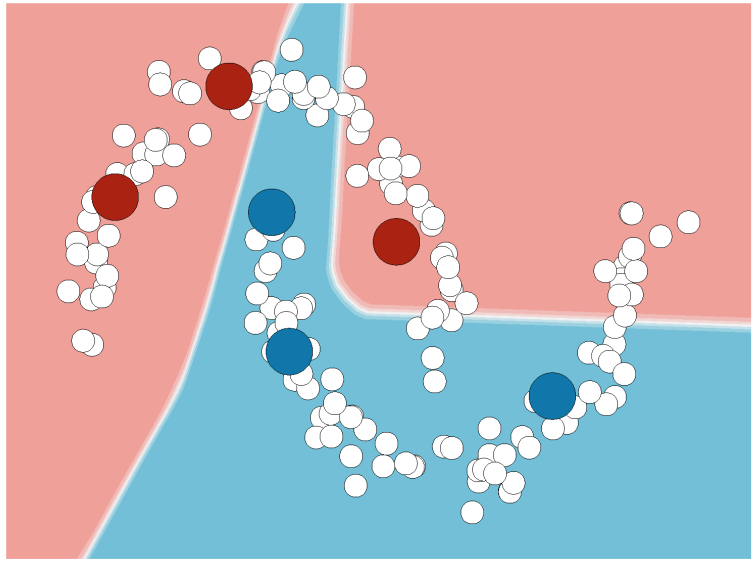
$$\hat{A} = \widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$$

$$\widehat{W}^{(0)}, \widehat{W}^{(1)} = \mathrm{argmin}_{W^{(0)},W^{(1)}} \ -\sum_{i \in Y_L} \log z_{i,y_i}(W^{(0)}, W^{(1)})$$

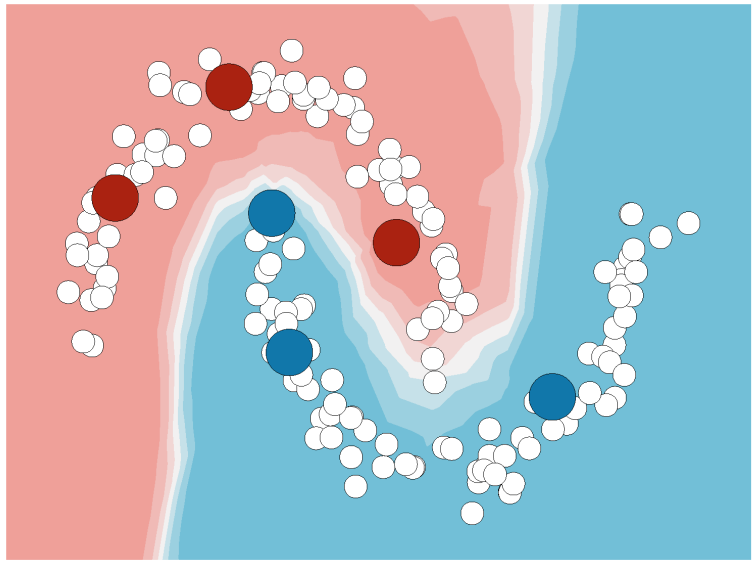
- Once the GCN parameters have been learned in this manner, they can be used to infer the labels of all nodes where labels not given

- Have explicitly leveraged the information in the graph (node connectivity) to enhance inference of unlabeled nodes

- Different than supervised learning, where we only train on the labeled samples $\{(x_i, y_i)\}_{i=1,N}$



Supervised Learning



Semi-Supervised Learning

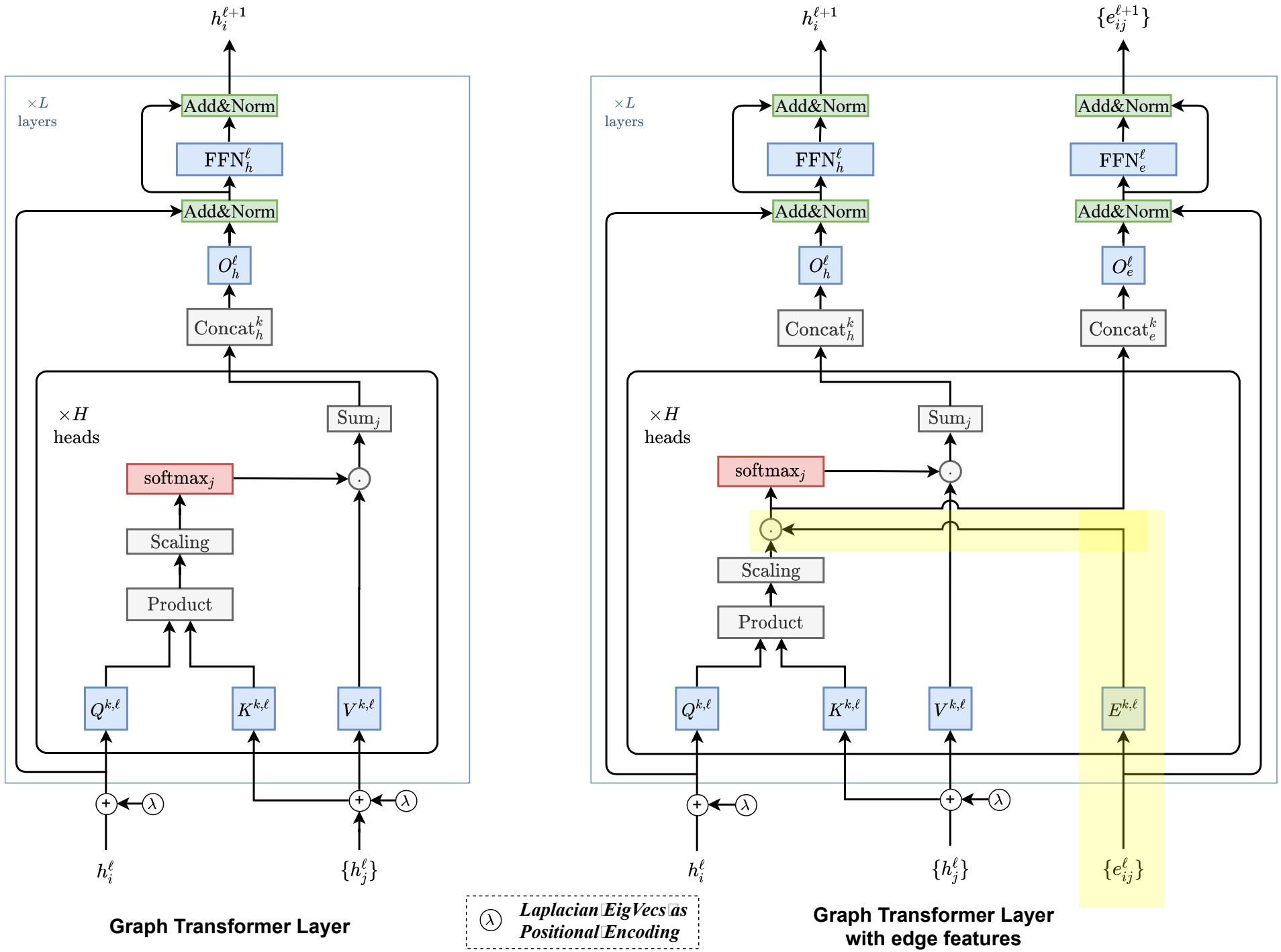
Connecting GCN and Transformer

$$Z = \mathrm{softmax}[\hat{A}\ \mathrm{ReLu}\left(\hat{A}XW^{(0)}\right)W^{(1)}]$$

$$\hat{A} = \widetilde{D}^{-1/2}\tilde{A}\widetilde{D}^{-1/2}$$

$$\widehat{W}^{(0)}, \widehat{W}^{(1)} = \mathrm{argmin}_{W^{(0)},W^{(1)}} \ -\sum_{i \in Y_L} \log z_{i,y_i}(W^{(0)}, W^{(1)})$$

- The GCN has many of the characteristics of a Transformer, like attention weights and value matrices
- Simpler and perhaps less powerful, as it effectively only has one attention head at each layer and no feedforward elements
- The Transformer can be used to address a similar form of semi-supervised learning, with the architecture discussed earlier



Graph Transformer Layer

Graph Transformer Layer with edge features

Relative Merits of Graph Transformer vs GCN

- Both models explicitly account for the graph structure via the graph Laplacian

- The GCN may have difficulty handling *vector* features on the observed edges (scalar weights handled by the Laplacian)

- The graph Transformer is a more-complicated model, but it has the advantage of explicitly handling vector features connected to observed edges

- Recent generalizations of the graph Transformer, like those discussed here, have demonstrated state-of-the-art results on analysis of data on a graph